

Product Requirements Document

Public web presence and authenticated service portal for industrial printing and signage machinery

This PRD defines the product scope, target users, features, requirements, and acceptance criteria for ElkaTech Launchpad: a public marketing and product-catalog website for industrial printing and signage machinery, combined with an authenticated service portal for creating and tracking machine service requests. The document is grounded in the repository README; anything not confirmed there is explicitly listed under Assumptions and Open Questions.

Document	Product Requirements Document (PRD)
Product	ElkaTech Launchpad
Version	1.0
Date	June 10, 2026
Status	Approved baseline — reflects the platform as built
Owner	ElkaTech Product Management
Audience	Stakeholders, investors, product, engineering, design
Classification	Internal / Confidential
Source of truth	Repository README.md (elkatech-launchpad)

Contents

1. Executive Summary	4
2. Product Vision	4
3. Problem Statement	4
3.1 Problems for customers	5
3.2 Problems for ElkaTech staff	5
4. Target Users	5
5. User Personas	5
5.1 Persona 1 — “Rajan”, print-shop owner (Customer)	6
5.2 Persona 2 — “Meera”, procurement researcher (Visitor)	6
5.3 Persona 3 — “Dev”, field service engineer (Engineer)	6
5.4 Persona 4 — “Anita”, service operations manager (Admin)	6
6. Business Goals	6
7. Product Goals	7
8. Non-Goals	7
9. Core Features	8
9.1 Public landing page	8
9.2 Product catalog pages	8
9.3 Theme system	9
9.4 Authentication	9
9.5 Customer approval workflow	9
9.6 Service request creation	9
9.7 Request tracking	9
9.8 Conversation and messages	10
9.9 Engineer queue	10
9.10 Admin dashboard	10
9.11 User management	10
9.12 Notification system	10
9.13 Health dashboard	10
10. User Stories by Role	11

10.1 Public visitor	11
10.2 Customer	11
10.3 Engineer	12
10.4 Admin	12
11. Functional Requirements	12
11.1 Public site	13
11.2 Identity and access	13
11.3 Service desk	14
11.4 Notifications and operations	14
12. Non-Functional Requirements	14
13. Access Control Requirements	15
14. Success Metrics / KPIs	16
15. MVP Scope	16
16. Future Scope	17
17. Risks and Constraints	17
18. Acceptance Criteria	18
18.1 Public site	18
18.2 Authentication and approval	18
18.3 Service workflow	18
18.4 Admin operations	19
19. Release Checklist	19
20. Assumptions and Open Questions	19

1. Executive Summary

ElkaTech Launchpad is the single web platform for ElkaTech's industrial printing and signage machinery business. It serves two tightly connected purposes: presenting the brand and product catalog to the public, and operating an authenticated service desk where approved customers raise machine service requests that engineers and administrators resolve.

The public half is a polished, responsive marketing site: a cinematic landing page with hero, about, work, brands, infrastructure, why-us, contact, and footer sections, plus seven dedicated product-category pages (solvent printers, UV printers, laser cutting machines, lamination machines, desktop UV printers, inkjet printers, and flatbed UV printers) with image carousels, specification tables, brochure links, and service-request entry points. Light and dark themes are supported with a persisted preference.

The protected half is a service portal rendered by the same frontend. Customers sign up with Firebase Authentication (email/password or Google), receive a server-side cookie session, and — once approved by an administrator — create service requests against catalog products. Requests carry a full conversation thread, internal staff notes, a strict status workflow, assignment to engineers, and a complete activity history. Email notifications keep customers and staff informed. Administrators manage user approvals, staff invitations, engineer assignment, and a live service-health dashboard.

The platform is in production today: it deploys to Vercel with serverless API bundles and a Neon-hosted PostgreSQL database. This PRD captures the product as built so stakeholders share one authoritative description, and so future scope can be planned against a stable baseline.

2. Product Vision

Industrial printing machinery is a high-consideration, relationship-driven business. Buyers research equipment carefully, and after purchase the quality of service support determines loyalty far more than the original sale. ElkaTech Launchpad exists to make both moments excellent:

- **Before the sale** — a premium digital storefront that presents machines with the confidence of an industrial brand: real specifications, brochures, imagery, and a clear path to contact ElkaTech.
- **After the sale** — a service experience that replaces scattered phone calls and email threads with a structured portal: every request is numbered, tracked, assigned, discussed in one thread, and visible to the customer at all times.

The long-term vision is one trusted destination where an ElkaTech customer can evaluate equipment, request support, follow progress, and communicate with the engineer working on their machine — with ElkaTech staff operating from the same system of record.

3. Problem Statement

3.1 Problems for customers

- Machine specifications, pricing context, and brochures for industrial printing equipment are hard to compare without a well-structured catalog presentation.
- Service issues on production machinery are urgent and costly; without a tracking system, customers cannot see whether their issue was received, who is working on it, or what its current status is.
- Conversations about a service issue fragment across channels, losing context (serial numbers, site location, contact phone, prior replies).

3.2 Problems for ElkaTech staff

- Engineers need a single queue of open work with clear ownership — who claimed a request, what state it is in, and what was already said to the customer versus internally.
- Administrators need control over who can use the portal: industrial service is a business relationship, so self-signed-up accounts must be reviewed and approved before they can create requests.
- Operations need confidence that the platform itself is healthy — service heartbeats, request load, and pending approvals in one view.

Context. The pre-platform baseline (e.g. requests handled by phone or email) is the implied motivation for the portal; the README documents the solution rather than the prior process. See Assumptions and Open Questions.

4. Target Users

User group	Who they are	Primary need	Access level
Public website visitors	Prospective buyers, researchers, and partners browsing the ElkaTech brand and product categories	Understand the company and compare machines: specs, images, brochures, price context	Anonymous — public routes only
Customers	Owners/operators of ElkaTech machinery with a portal account	Create service requests, follow status, converse with the assigned engineer	Authenticated, role <i>customer</i> ; must be admin-approved to create requests
Engineers	ElkaTech field/service engineers invited by an administrator	Work a queue: claim requests, reply to customers, add internal notes, drive status to resolution	Authenticated, role <i>engineer</i> ; queue and assigned work
Admins	ElkaTech service managers / platform operators	Approve customers, invite staff, assign engineers, oversee load and platform health	Authenticated, role <i>admin</i> ; full portal access including dashboard and user management

5. User Personas

Illustrative. Personas are illustrative composites created for this PRD to make the four user groups concrete. They are consistent with the README's roles and workflows but are not sourced from user research.

5.1 Persona 1 — “Rajan”, print-shop owner (Customer)

- Runs a mid-size signage and large-format print shop with two ElkaTech machines on the floor.
- Downtime on a solvent printer costs him client deadlines; when a machine fails he wants to report it in minutes — with the serial number, site location, and a photo-ready description — and then see movement: triaged, assigned, in progress.
- Comfortable with Google sign-in; appreciates that he can check status from his phone instead of calling.

5.2 Persona 2 — “Meera”, procurement researcher (Visitor)

- Evaluating UV flatbed and laser cutting options for a packaging company; compares specification tables and downloads brochures.
- Judges vendors partly by the quality of their web presence; a fast, professional, dark-mode-friendly site signals an organized company.
- Converts via the contact section or the service/sales CTA on a product page.

5.3 Persona 3 — “Dev”, field service engineer (Engineer)

- Starts the day in the portal queue; claims requests that match his territory and skills.
- Needs the full picture on one screen: customer description, product snapshot, phone, location, prior conversation, and internal notes from colleagues that the customer must not see.
- Updates status as work progresses and expects the system to prevent invalid jumps (e.g. closing a request that was never triaged is an explicit, deliberate action).

5.4 Persona 4 — “Anita”, service operations manager (Admin)

- Reviews pending customer signups every morning and approves genuine ElkaTech customers; rejects or suspends doubtful accounts.
- Invites new engineers, assigns or reassigns requests when load is uneven, and reopens resolved requests when a customer reports the issue recurred.
- Glances at the admin dashboard for approval counts, request load, recent signups, and live service heartbeats.

6. Business Goals

Framing. The README documents capabilities, not commercial targets. The goals below state the business intent each capability serves; numeric targets belong to the KPI section and are proposed, not confirmed.

- **BG-1 — Premium digital presence.** Present ElkaTech as a credible industrial machinery brand through a polished, responsive, themed marketing site that supports the sales conversation.
- **BG-2 — Catalog-driven demand.** Let prospects self-serve product research (specs, brochures, imagery) and route them to contact and service CTAs.
- **BG-3 — Structured after-sales service.** Convert ad-hoc service communication into tracked, assignable, auditable requests, improving resolution discipline and customer trust.
- **BG-4 — Controlled access.** Keep the portal limited to verified, admin-approved customers so service capacity is spent on real ElkaTech relationships.
- **BG-5 — Operational self-sufficiency.** Give ElkaTech staff built-in tools (queue, dashboard, health monitoring, user management) so daily operations need no external systems.
- **BG-6 — Low operating cost.** Run on managed infrastructure (Vercel serverless, Neon PostgreSQL free/low tiers) with no dedicated ops team.

7. Product Goals

- **PG-1.** A visitor can reach any product category, view machine specifications and brochures, and find a service or contact CTA within two clicks of the landing page.
- **PG-2.** A new customer can sign up (email/password or Google), verify their email, and — once approved — submit a complete service request in a single guided form.
- **PG-3.** Every service request has, at all times: a unique request number, an unambiguous status from a fixed workflow, an assignment state, a conversation thread, and a complete activity history.
- **PG-4.** Engineers and admins can see and act on all open work from one queue, with role-appropriate actions only.
- **PG-5.** Customers and staff receive email notifications for the key moments (e.g. signup events, request created, request claimed/assigned, status changes, replies) without manual effort.
- **PG-6.** Admins can fully manage the user base — approve, reject, suspend, reactivate, invite staff, and remove users — from the portal.
- **PG-7.** The portal looks and behaves like one product: shared graphite/ivory/copper design system, stable animations, dark/light themes, and consistent components across all roles.

8. Non-Goals

Explicitly out of scope for the current product:

- **E-commerce.** No cart, checkout, payments, quotes, or order management. Product pages show display prices and brochures only.

- **Inventory / parts management.** No stock, spare-parts ordering, or logistics features.
- **Content management system.** Public product pages are maintained by developers from local component data plus a seeded catalog, not by a CMS.
- **Native mobile apps.** The product is a responsive web application only.
- **Anonymous service requests.** Only authenticated, email-verified, admin-approved customers can create requests.
- **SLA automation.** No automatic escalation timers, SLA clocks, or penalty tracking; priority is informational.
- **Third-party monitoring stack.** The health dashboard is sourced entirely from internal service heartbeats; there is deliberately no AWS CloudWatch dependency.

9. Core Features

9.1 Public landing page

A responsive marketing landing page composed of hero, about, work, brands, infrastructure, why-us, contact, and footer sections. The hero uses cinematic headline motion; the rest of the page uses the shared `StableReveal` animation primitive so sections fade in without layout shift. An initial branded loader animation (`IntroAnimation`) plays on first load. The shared `SiteHeader` navbar adapts to light and dark page contexts and to scroll position.

9.2 Product catalog pages

Seven dedicated category routes present ElkaTech's machine families: solvent printers, UV printers, laser cutting machines, lamination machines, desktop UV printers, inkjet printers, and flatbed UV printers (plus a legacy redirect from `/inject-printer` to `/inkjet-printer`). Each category page is composed from a shared `ProductCategoryPage` layout providing:

- Product image carousels backed by static assets.
- Specification tables per machine.
- Display price and brochure download links.
- Highlights and a theme-aware decorative background (`ProductPageBackground`).
- Service-request entry points that lead into the portal.

Known constraint. Catalog data currently exists in two parallel sources: local arrays inside the public page components, and a database-backed catalog seeded from shared contracts for the portal's product selector. They are not automatically synchronized; content updates must touch both.

9.3 Theme system

ThemeProvider supports light, dark, and system modes, persists the choice in localStorage (key elkatech-ui-theme), and switches a root CSS class. The navbar exposes light/dark variants so dark landing sections and light category pages both stay readable. The protected portal uses a unified graphite/ivory/copper design system across all pages.

9.4 Authentication

Firebase Authentication is the identity provider for email/password and Google sign-in. The frontend obtains a Firebase ID token and exchanges it at the gateway for a server-side cookie session; CSRF protection and role-based access control are enforced on every mutating call. Firebase handles verification and password-reset emails for Firebase-issued accounts; legacy backend flows (signup, login, verify-email, reset) remain available for pre-Firebase accounts, as does a legacy server-side Google OAuth flow.

9.5 Customer approval workflow

Every new public signup is created with role customer and approval status pending_approval. Pending users can sign in and look around, but cannot create service requests; a polished approval-state card explains the situation. Admins approve, reject, suspend, or reactivate accounts from the user management page. Approval is independent of email verification: Firebase confirms identity, an ElkaTech admin confirms the business relationship.

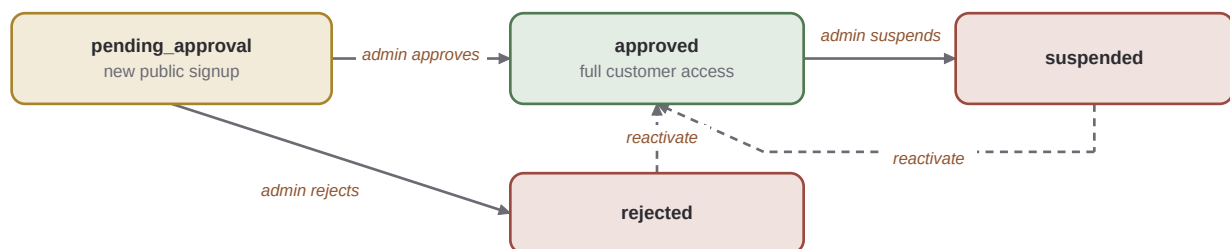


Figure 1 — Customer approval lifecycle. Staff invited by an admin start directly as approved.

9.6 Service request creation

Approved customers create requests from a guided form at /app/requests/new. A request captures: the catalog product (with a frozen product snapshot), subject, description, contact phone, site location, optional serial number, and a priority (low, normal, high, urgent). Submission requires an authenticated session, verified email, and approved status; the gateway returns specific error codes (USER_PENDING_APPROVAL, USER_REJECTED, USER_SUSPENDED) otherwise.

9.7 Request tracking

Customers see their own requests at /app/requests with status badges; engineers see assigned work; admins see everything. Every request moves through a strict workflow — new, triaged, assigned, in_progress, waiting_for_customer, resolved, closed — with only valid transitions offered in the UI and enforced by the backend. Reopening resolved/closed requests is admin-only.

9.8 Conversation and messages

Each request carries a chat-style conversation. Customer-visible messages and internal staff notes share the thread but are clearly separated: internal notes are badge-labelled and never shown to customers. Messages display real author names (with email and role fallbacks), role chips for staff, and right/left alignment for own versus others' messages. A composer with a visibility selector sits attached to the thread.

9.9 Engineer queue

Engineers and admins work from `/app/queue`, which lists open and assigned requests. An engineer claims an unassigned request to take ownership; claiming an already-assigned request is cleanly refused. Admins can assign or reassign any engineer explicitly. Assignment and reassignment are recorded as distinct history events.

9.10 Admin dashboard

`/app/admin` gives administrators a live operational picture: approval status counts, service-request load, recent signups and activity, and per-service health heartbeats (healthy / degraded / down with latency), refreshing every 60 seconds.

9.11 User management

`/app/users` lets admins:

- Review all users with role, approval status, account origin, and verification state.
- Approve, reject, suspend, or reactivate customers.
- Invite engineers and admins by email (invited accounts arrive approved).
- Remove users. The remove action permanently deletes the user and their service-request data in one transaction; admin accounts are protected from removal, and the confirmation dialog warns that deletion is permanent.

A separate My Account page (`/app/account`) lets every signed-in user view their profile, edit their display name, and trigger a password reset email for their own address. Email changes are deliberately admin-handled because Firebase is the identity source.

9.12 Notification system

Auth and service-desk actions emit events into transactional outbox tables; the notification service converts them into emails and records every delivery. Two production flows (account-added invitation and request claimed/assigned) are sent through AWS SES templates; all other flows — and all local development via Mailpit — use SMTP. Failed sends are recorded per recipient with the error, so one failing address does not block other recipients.

9.13 Health dashboard

Every service exposes a heartbeat endpoint; the gateway aggregates them into a per-service healthy / degraded / down snapshot with latency measurements, surfaced on the admin dashboard. This is intentionally self-contained — no external monitoring vendor is required for day-to-day visibility.

10. User Stories by Role

10.1 Public visitor

ID	User story	Priority
V-1	As a visitor, I can browse a landing page that explains who ElkaTech is and what they sell, so that I can judge the company quickly.	Must
V-2	As a visitor, I can open a category page for each machine family and view images, specifications, prices, and brochures, so that I can compare equipment.	Must
V-3	As a visitor, I can use the site comfortably on mobile and in dark or light mode, so that the experience matches my context.	Must
V-4	As a visitor, I can follow a contact or service CTA, so that I can start a conversation with ElkaTech.	Must

10.2 Customer

ID	User story	Priority
C-1	As a new customer, I can sign up with email/password or Google and verify my email, so that I get a portal account.	Must
C-2	As a pending customer, I can sign in and see a clear explanation that my account awaits approval, so that I know why I cannot create requests yet.	Must
C-3	As an approved customer, I can create a service request with product, subject, description, phone, location, serial number, and priority, so that ElkaTech has everything needed to help.	Must
C-4	As a customer, I can see all my requests with their current status, so that I always know where things stand.	Must
C-5	As a customer, I can read and reply in the request conversation and see who is assigned, so that I communicate in context.	Must
C-6	As a customer, I receive emails at key moments (request created, picked up by an engineer), so that I do not need to poll the portal.	Must
C-7	As a customer, I can reset my password and edit my display name, so that I can maintain my own account.	Should

10.3 Engineer

ID	User story	Priority
E-1	As an engineer, I can view the queue of open and assigned requests, so that I can pick up work.	Must
E-2	As an engineer, I can claim an unassigned request and get a clear refusal if someone claimed it first, so that ownership is unambiguous.	Must
E-3	As an engineer, I can reply to the customer or add an internal note invisible to them, so that I communicate appropriately.	Must
E-4	As an engineer, I can move a request through valid workflow statuses only, so that the lifecycle stays meaningful.	Must
E-5	As an engineer, I see the customer's product snapshot, contact phone, site location, and serial number on the request, so that I can prepare the intervention.	Must

10.4 Admin

ID	User story	Priority
A-1	As an admin, I can approve, reject, suspend, or reactivate customer accounts, so that portal access stays controlled.	Must
A-2	As an admin, I can invite engineers and admins by email, so that staff onboarding is self-service.	Must
A-3	As an admin, I can assign or reassign any request to any engineer, so that load stays balanced.	Must
A-4	As an admin, I can reopen resolved or closed requests, so that recurring issues resume their history.	Must
A-5	As an admin, I can see approval counts, request load, recent activity, and live service heartbeats on one dashboard, so that I can run operations at a glance.	Must
A-6	As an admin, I can permanently remove a user and their request data, with admin accounts protected and an explicit warning, so that test or invalid accounts do not linger.	Should

11. Functional Requirements

Requirements are numbered FR-x. “Must” items are implemented and required for any release; “Should” items are implemented conveniences that could be temporarily degraded without breaking the core proposition.

11.1 Public site

ID	Requirement	Priority
FR-1	The system shall serve a public landing page with hero, about, work, brands, infrastructure, why-us, contact, and footer sections.	Must
FR-2	The system shall serve seven product-category pages with carousels, specification tables, display prices, brochure links, and service CTAs, plus a redirect from the legacy /inject-printer path.	Must
FR-3	The system shall support light, dark, and system themes with a persisted preference and no flash of incorrect theme on reload.	Must
FR-4	Entrance animations shall not cause layout shift and shall respect prefers-reduced-motion.	Must

11.2 Identity and access

ID	Requirement	Priority
FR-5	The system shall support signup and login via Firebase email/password and Google, exchanging the Firebase ID token for a cookie session.	Must
FR-6	The system shall maintain legacy email/password, verify-email, password-reset, and server-side Google OAuth flows for pre-Firebase accounts.	Should
FR-7	New public signups shall start as role customer with approval status pending_approval.	Must
FR-8	The system shall block request creation for users who are not approved, returning USER_PENDING_APPROVAL, USER_REJECTED, or USER_SUSPENDED accordingly.	Must
FR-9	Sessions shall be cookie-based with CSRF tokens required on all mutating requests.	Must
FR-10	Admins shall be able to approve, reject, suspend, reactivate, invite (engineer/admin), and remove users.	Must
FR-11	Users shall be able to view their account details, update their display name, and request a password reset for their own address from a My Account page.	Should

11.3 Service desk

ID	Requirement	Priority
FR-12	Approved customers shall create requests with product, subject, description, contact phone, site location, optional serial number, and priority; the system assigns a unique request number and freezes a product snapshot.	Must
FR-13	Request listing shall be role-aware: customers see their own, engineers see queue/assigned scope, admins see all.	Must
FR-14	The request workflow shall enforce the strict status transition map, reject same-status updates, and restrict reopening to admins.	Must
FR-15	Engineers/admins shall claim unassigned requests; a second claim attempt shall be refused with a clear conflict message; admins shall assign or reassign explicitly, and reassignment shall be recorded as a distinct event.	Must
FR-16	Messages shall support customer_visible and internal_note visibility; internal notes shall never be exposed to customers.	Must
FR-17	Every request shall carry a complete activity history (created, claimed, assigned, reassigned, status changes and related events) with actor and timestamp.	Must
FR-18	Request detail shall resolve participant display names/emails so all roles see real names without needing admin endpoints.	Should

11.4 Notifications and operations

ID	Requirement	Priority
FR-19	Auth and service-desk events shall be written to transactional outbox tables in the same database transaction as the state change.	Must
FR-20	The notification service shall convert outbox events to emails, recording one delivery row per recipient with sent/failed status, attempts, and last error.	Must
FR-21	Account-added invitations and request-claimed notifications shall be sent via AWS SES templates in production; other flows and local development shall use SMTP (Mailpit locally).	Must
FR-22	Each service shall expose a health endpoint; the gateway shall aggregate per-service healthy/degraded/down status with latency for the admin dashboard, refreshed on a 60-second cadence.	Must
FR-23	The admin dashboard shall show approval counts, request load, and recent signups/activity.	Must

12. Non-Functional Requirements

Category	Requirement
Usability	The portal presents one coherent design system (graphite/ivory/copper) across all roles; status is always visible via badges; blocked states (pending approval, unverified email) are explained in-place with polished cards and banners rather than raw errors.
Accessibility	Reduced-motion preferences are respected; light/dark variants maintain readable contrast; UI is built on Radix primitives which provide accessible behavior for interactive components.

Category	Requirement
Responsiveness	All public pages and the portal (including the collapsible sidebar shell) work on mobile, tablet, and desktop viewports.
Performance	The frontend is a static Vite build served by Vercel's CDN; API routes run as serverless functions; the catalog service supports an optional Redis-compatible cache. Reveal animations are designed to avoid refresh-time layout shift.
Reliability	Email delivery uses the outbox pattern so notifications are never lost to a crashed process; deliveries are idempotent per (event, recipient); per-recipient failures do not block other recipients.
Security	Cookie sessions, CSRF tokens on mutations, RBAC at the gateway, internal-token gating between services, hashed session/auth tokens, and admin approval before customers gain request-creation rights. Secrets (Firebase private key, tokens, cookies) are never logged.
Maintainability	npm-workspace monorepo with shared TypeScript/Zod contracts; ESLint and Vitest are in place (with a known pre-existing lint baseline to clean up).
Portability	Local development runs the full platform with Docker (Postgres + Mailpit) and Node 22; production runs on Vercel + Neon without code changes.

Assumption / open question. The README defines no numeric targets for latency, uptime, or capacity. Concrete SLOs (e.g. p95 API latency, monthly uptime) remain to be agreed and should be added to this section once measured in production.

13. Access Control Requirements

Three roles exist: customer, engineer, admin. Access is enforced at the gateway (sessions, CSRF, role checks, approval checks) and mirrored in the frontend (ProtectedRoute, role-aware navigation).

Capability	Visitor	Customer	Engineer	Admin
Browse public site and catalog	Yes	Yes	Yes	Yes
Sign up / log in	Yes	—	—	—
View own requests	No	Yes	Yes (assigned)	Yes (all)
Create service request	No	Yes (approved + verified email only)	No ¹	No ¹
View queue (/app/queue)	No	No	Yes	Yes
Claim request	No	No	Yes	Yes
Assign / reassign engineer	No	No	No	Yes
Change request status	No	No	Yes	Yes
Reopen resolved/closed	No	No	No	Yes
Post customer-visible message	No	Yes (own requests)	Yes	Yes
Post internal note	No	No	Yes	Yes

Capability	Visitor	Customer	Engineer	Admin
User management (/app/users)	No	No	No	Yes
Admin dashboard + health	No	No	No	Yes
My Account page	No	Yes	Yes	Yes

¹ Request creation is a customer action; the README scopes the creation form and approval gating to customers. Staff manage rather than open requests.

- Approval gating: POST /api/requests is additionally gated on approval status approved and verified email, independent of role checks.
- Route gating: /app/queue requires engineer or admin; /app/admin and /app/users require admin; all /app routes require a session.
- Error behavior: permission failures return 403 with specific codes where defined (USER_PENDING_APPROVAL, USER_REJECTED, USER_SUSPENDED); the UI renders friendly blocked-state cards.

14. Success Metrics / KPIs

Proposed. No KPIs are defined in the README. The following metrics are proposed for adoption; all are measurable from existing data (requests, history, deliveries, users) without new instrumentation. Targets are placeholders to be set by the business.

Metric	Definition	Source	Initial target
Approval turnaround	Median time from signup to admin approval decision	auth.users timestamps	< 1 business day
First response time	Median time from request creation to first staff message or claim	request history/messages	< 4 business hours
Resolution time	Median time from new to resolved	request history	Set after baseline
Requests via portal	Share of service issues arriving as portal requests vs. other channels	requests count vs. manual tally	> 80%
Notification health	Delivery success rate (sent vs. failed)	notification.deliveries	> 99%
Service availability	Share of health probes returning healthy	admin health snapshots	Set after baseline
Catalog engagement	Visits to category pages and brochure clicks	web analytics (to be added)	Set after baseline

15. MVP Scope

The MVP is the platform as currently built and deployed. It comprises:

- Public landing page and seven product-category pages with theming and stable animation.
- Firebase authentication (email/password + Google) bridged to cookie sessions with CSRF and RBAC; legacy auth flows retained.
- Customer approval workflow with pending/approved/rejected/suspended states and admin controls.
- Service request creation, role-aware tracking, strict status workflow, claim/assign/reassign, conversation with internal notes, and full activity history.
- Email notifications via outbox pattern (SES templates for invite and claim flows; SMTP otherwise; Mailpit locally).
- Admin dashboard (approvals, load, recent activity, heartbeats), user management, staff invitations, user removal, My Account page.
- Vercel + Neon production deployment with bundled serverless API handlers.

Explicitly not in the MVP: everything listed under Non-Goals, plus the items in Future Scope below.

16. Future Scope

Documented as future improvements in the repository:

- Reconcile or generate the duplicate public-page and seeded catalog datasets from one source of truth.
- Automate serverless API bundling in package scripts or CI instead of manual regeneration.
- Expand automated test coverage beyond the current workflow-focused tests.
- Formalize and document production observability, backup, and operational procedures.
- Pin the Node version for reproducible local and CI environments.

Additional product directions consistent with the platform (not yet committed):

- File/photo attachments on service requests and messages.
- In-portal notification center complementing email.
- Customer-facing knowledge base or machine documentation area.
- Reporting exports for request volume and resolution metrics.

17. Risks and Constraints

Risk / constraint	Impact	Mitigation
Dual catalog sources (component data vs. seeded DB) drift apart	Public pages and portal selector disagree; customer confusion	Documented update procedure touching both; future single-source generation
Committed serverless bundles (api/*.mjs) must be regenerated after backend changes	Production silently runs stale code	Documented bundling step; candidate for CI automation
Manual production migrations (run from a laptop against Neon)	Schema drift between code and database	Documented one-time commands; migration discipline; future CI

Risk / constraint	Impact	Mitigation
Admin approval is a human bottleneck	New customers wait; first impression risk	Dashboard surfaces pending counts; proposed approval-turnaround KPI
Pre-existing lint failures in shared UI/config files	Noise hides new issues	Documented; cleanup planned
Single shared production database for previews (Neon branch-per-deploy disabled)	Preview testing touches real data	Deliberate choice for migration sanity; care required in previews
No LICENSE file; ownership terms unconfirmed	Blocks external publication	Confirm ownership/redistribution before any external release
Email deliverability depends on SES/SMTP configuration (sandbox limits, verified senders)	Missed notifications	Delivery log with per-recipient errors; SES production access

18. Acceptance Criteria

18.1 Public site

- Given any of the seven category routes, when the page loads, then images, specs, price display, brochure link, and a service CTA render without layout shift, in both themes, on mobile and desktop.
- Given /inject-printer, when visited, then the browser is redirected to /inkjet-printer.
- Given a theme choice, when the page is reloaded, then the same theme is applied from the persisted preference.

18.2 Authentication and approval

- Given a new email/password signup, when it completes, then a verification email is sent, a session is established, and the account is role customer with status pending_approval.
- Given a pending customer on /app/requests/new, when the page loads, then the approval-state card is shown instead of the form, and POST /api/requests returns USER_PENDING_APPROVAL.
- Given an admin approves the user, when the customer retries, then request creation succeeds.
- Given a Google sign-in, when the popup completes, then one click suffices — the user lands in the portal without re-clicking.
- Given any mutating call without a valid CSRF token, when submitted, then the gateway rejects it.

18.3 Service workflow

- Given an approved customer submits a valid request, when it is created, then it has a unique request number, status new, a product snapshot, a request_created history event, and a confirmation email is delivered.
- Given an unassigned request, when an engineer claims it, then it is assigned to them; a second claim by another engineer returns a 409 conflict with a friendly message.

- Given any status-change attempt, when the transition is not in the allowed map (or is a same-status update), then the backend rejects it with 400 'Invalid workflow transition.' and the UI never offered it.
- Given a resolved or closed request, when a non-admin attempts to reopen, then the action is forbidden; an admin reopen sets status new and is recorded in history.
- Given an internal note, when a customer views the request, then the note is absent from their conversation view.

18.4 Admin operations

- Given the admin dashboard, when loaded, then approval counts, request load, recent activity, and per-service heartbeats with latency are visible and refresh on a 60-second cadence.
- Given a staff invitation, when accepted, then the account arrives with the invited role and approved status, and an account-added email was delivered.
- Given a user removal, when confirmed, then the user and their request data are permanently deleted in one transaction, admin accounts cannot be removed, and the list updates immediately.

19. Release Checklist

1. All backend changes bundled: node scripts/bundle-api.cjs run and api/*.mjs committed.
2. Database migrations applied to the production (Neon) database; seed and bootstrap-admin run where required.
3. Vercel environment variables present for Production and Preview: Firebase web + Admin SDK values, INTERNAL_SERVICE_TOKEN (strong random), SMTP settings, SES settings (region, sender, template names), bootstrap admin credentials, POSTGRES_URL injected via the Neon integration.
4. Firebase Authorized Domains include the production domain (and preview pattern if testing previews).
5. Neon integration configured with POSTGRES variable prefix and branch-per-deployment disabled for both Preview and Production.
6. Vercel Deployment Protection set to 'Only Production Deployments' or disabled, so internal service-to-service calls succeed.
7. Build, typecheck, and tests pass (npm run build, tsc --noEmit, npm run test); lint reviewed against the known pre-existing baseline.
8. Smoke test on the deployed URL: landing + one category page render in both themes; signup → pending card; admin approve → request create → claim → status change → emails visible in delivery log.
9. Admin health dashboard shows all services healthy.
10. Rollback plan: previous Vercel deployment retained for instant re-promotion; database migrations reviewed for reversibility.

20. Assumptions and Open Questions

- **Prior process baseline.** The README does not describe how service requests were handled before the portal; the problem statement assumes phone/email ad-hoc handling.
- **Personas and KPIs.** Personas are illustrative; all KPI targets are proposals pending business sign-off.
- **SLAs.** Priority levels exist but no response/resolution SLAs are defined; decide whether priorities should drive commitments.
- **Catalog governance.** Who owns product content updates, and when will the dual catalog sources be unified?
- **Analytics.** No web analytics are documented; catalog engagement KPIs require adding a measurement tool.
- **Ownership/license.** No LICENSE file exists; ownership and redistribution terms must be confirmed before external publication.
- **User removal semantics.** The platform recently moved from soft-deleting removed users to permanent hard deletion with protected admin accounts; confirm this matches data-retention policy and customer-data obligations.
- **Email provider strategy.** Two transactional flows use AWS SES templates while others use SMTP; confirm the long-term single provider direction.